# Module 1: Accounts and Programs

## Core Concepts in Solana

# Accounts vs Programs

## Solana's Account Model

- **Accounts**: Data storage containers (like database records)
- **Programs**: Executable code (like smart contracts)
- **Key Difference**: Accounts can hold both data AND code

## EVM Comparison

- **EVM**: Contracts hold code, separate storage
- **SVM**: Accounts are unified - they can be programs OR data

# Account Types

## 1. User Accounts (External Wallets)

- Controlled by private keys
- Hold SOL and tokens
- Can sign transactions
- Created by System Program

## 2. Program Accounts

- Contain executable bytecode
- Immutable once deployed
- Executed by Solana runtime
- Cannot sign transactions directly

# Account Types (continued)

## 3. Program Derived Addresses (PDAs)

- Deterministically derived from seeds + program ID
- No private key exists
- Controlled by programs
- Enable program-controlled accounts

## 4. System Accounts

- Built-in Solana programs
- System Program, SPL Token Program, etc.
- Handle core blockchain operations

# Account States

## Account Lifecycle

- **Uninitialized**: No data, zero balance
- **Active**: Contains data, has balance
- **Frozen**: Temporarily locked
- **Closed**: Zero balance, can be garbage collected

## Rent Exemption

- Accounts must maintain minimum balance
- Prevents storage spam
- "Rent" paid to validators for storage

# Architecture Overview

```
API Layer
  AccountsController
  • POST /accounts → create()
  • GET /accounts → findAll()
  • GET /accounts/:id → findOne()
  • GET /accounts/address/:address → findByAddress()
  • PUT /accounts/:id → update()
  • DELETE /accounts/:id → remove()
  • GET /accounts/info/:address → getAccountInfo()
  • GET /accounts/balance/:address → getBalance()
        │
        ▼
Service Layer
  AccountsService
  • create() ← API calls
  • findAll() ← API calls
  • findOne() ← API calls
  • findByAddress() ← API calls
  • update() ← API calls
  • remove() ← API calls
  • getAccountInfo() ← API calls
  • getBalance() ← API calls
        │
        ▼
Data Layer
  Account Entity
  • id: string (PrimaryGeneratedColumn)
  • address: string (Unique)
  • owner: string (Nullable)
  • balance: bigint (Default 0)
  • isPda: boolean (Default false)
  • programId: string (Nullable)
  • metadata: jsonb (Nullable)
  • createdAt: Date (CreateDateColumn)
  • updatedAt: Date (UpdateDateColumn)
        │
        ▼
External Integrations
  Solana Web3.js
  • Connection → getAccountInfo()
  • PublicKey → getBalance()

Database
  PostgreSQL + TypeORM
  • Repository<Account> ← Service Layer
```

# API Endpoints

## Account Management

- `POST /accounts` - Create new account record
- `GET /accounts` - List all accounts
- `GET /accounts/:id` - Get account by ID
- `GET /accounts/address/:address` - Get account by Solana address
- `PUT /accounts/:id` - Update account metadata
- `DELETE /accounts/:id` - Remove account record

## Blockchain Queries

- `GET /accounts/info/:address` - Get on-chain account info
- `GET /accounts/balance/:address` - Get SOL balance

# Database Schema

## Account Entity Fields

- `id` : Primary key (UUID)
- `address` : Solana public key (unique)
- `owner` : Account owner address
- `balance` : SOL balance (bigint)
- `isPda` : Program Derived Address flag
- `programId` : Associated program ID
- `metadata` : Additional JSON data
- `createdAt/updatedAt` : Timestamps

# Key Implementation Concepts

## PDA Generation

```
// Derive PDA from seeds and program ID
const [pdaAddress, bump] = PublicKey.findProgramAddressSync(
  [Buffer.from("escrow"), userPublicKey.toBuffer()],
  programId
);
```

## Account Info Retrieval

```
// Get account information from blockchain
const accountInfo = await connection.getAccountInfo(publicKey);
if (accountInfo) {
  // Account exists with data
  console.log('Balance:', accountInfo.lamports);
  console.log('Owner:', accountInfo.owner.toString());
}
```

# Common Patterns

## Account Creation Flow

1. Generate keypair or derive PDA
2. Check if account exists
3. Create account record in database
4. Return account information

## Balance Checking

1. Query Solana RPC for account info
2. Convert lamports to SOL
3. Cache result for performance
4. Return formatted balance

# Security Considerations

## Access Control

- Validate account ownership
- Implement proper authorization
- Rate limit API calls

## Data Validation

- Verify Solana addresses
- Sanitize metadata input
- Prevent SQL injection

# Next Steps

## Module 1 Implementation Tasks

- ✅ Basic account CRUD operations
- ✅ Solana RPC integration
- ⏳ PDA generation service
- ⏳ Account abstraction features

## Related Modules

- **Module 2**: Transactions & Instructions
- **Module 3**: Token Standards
- **Module 4**: Account Abstraction

# Thank You!

## Questions?

**Ready to explore Transactions & Instructions?**

Next: Module 2 - Transactions & Instructions ➡️